

① Object Oriented Programming

Concepts -

=> Java is fundamentally an object oriented programming language.

① In Procedural Programming

we think in terms of

Data + functions.

② In OOPs, we organize them around objects.

Object

also known as (State / Behavior) also known as.

fields / methods.

for ex -> considering a bank account.

its states could be - account no.

account holder

balance.

② its behavior could be = deposit()
withdrawal()
displayBalance()
③
we can model it as follows

class BankAccount {
 String accountNo; // id
 String accountHolder;
 double balance;
 void deposit(double amount);
}

behavior
 balance += amount;
 void withdraw(double amount)
 balance -= amount;

Note -> Just an example, don't think about the syntax, understand the example.

Class - It is a blueprint / template from which objects are created.

3) for ex -> Class Student {

int rollNo;

String name;

double marks;

-> the class describes what a

student object will contain.

-> it does not represent one parti-

- cular student.

Object -> An object is an instance

of a class.

object
creation

for ex -> Student s1 = new Student();

reference variable

Class

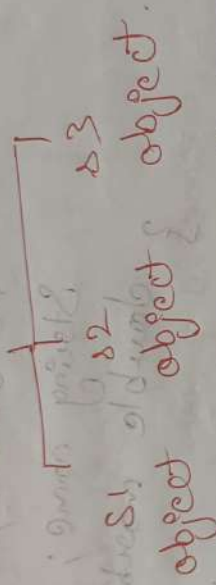
refers to the object

Note -> We can create many objects from the same class.

-> each object has its own data.

4) Question → Student s1 = new Student();
 Student s2 = new Student();
 Student s3 = new Student();

Student Class



Properties of a class -

- Collection of objects
- Not a real world entity
- Does not occupy memory.

Syntax → access modifier Class Class Name

? if not provided

its default

{ access modifier

abhi? age batanyu

thoda sunke }

- methods

- fields

- constructors

- nested class

⑤ Object Properties -

- > instance of a class
- > real world entity
- > occupies memory.

Methods -

A set of codes which performs a particular task.

- Advantages -
- Code reusability
 - Code optimization.

Syntax -

access - modifier return type

(method name (list of parameters)

{

task / set of instructions

;

Ex -> public void display (int a)

ye hai access modifier.

ye hai task / set of instructions

-> kya hata hai aage bataunga

⑥ How to create an object using new keyword

① Declaration -
e.g. → Animal tom; → reference to class name object

② Instantiate -
tom = new

③ Initialization
→ object creation

tom = new Animal();
or in one line

{ Animal tom = new Animal(); }

Calling method using object

tom → tom.sun();

① Operator is used to call the method.

① Full example

```
class Animal {  
    public void cat() {  
        System.out.println("xyz");  
    }  
    public static void main (String [] args)
```

① System.out.print(" ");

② Animal tom = new Animal();

③ tom.cat();

O/p -> xyz

Q-7 Class Student {

int rollno;
String name;
public static void main(
String[] args)

3 types we

have the

↓

grad class

abhi

(imp hai)

Student s1 = new Student();

s1.rollno = 101;

s1.name = "Akash";

}
→ initializing the values

the using not
assigning values to
the variables belonging

to s1

Student s2 = new Student();

s2.rollno = 101;

s2.name = "Aman";

assigning

values to variables belonging to s2

Q) Note $\rightarrow$ s1 and s2 are reference variables that refers to a Student object

like this $\rightarrow$ s1 $\rightarrow$ reference variable

$\rightarrow$ Student object

rollno = 101
name = "Akash"

$\rightarrow$ just like i told you in Strings topic.

null Reference - A reference can contain null.

Ex $\rightarrow$ Student s = null;

$\rightarrow$ means s currently refers to no object.

Object Dereferencing -

String s = new Student();

s = null;

$\rightarrow$ The object still exists but now no reference from s to it.

$\rightarrow$ so now eligible for garbage collection

10) Garbage Collection - Automatic reclamation of memory occupied by objects that are no longer reachable!

Java may eventually reclaim its memory.

Therefore A - garbage! This is #

then return to no

then = a thread? 5-0

before it's called & return

to go on at

then return to no

(1) thread? no & private

Methods

⑪

- Collection of statements grouped together to perform an operation
- Methods can be used to define reusable code and organize and simplify code.

Defining a Method

- `modifier methodtype methodname (list of parameters)`

```
{  
    // method body.  
}
```

e.g. → `public static int sum (int num1, int num2)`
method header {
 int result;

```

12) if (num1 > num2)
    result = num1;
else
    result = num2;
return result;

```

What's in method header

① → access modifier (public, private, protected, default)

② → method type

static or non-static

③ → return value type → int, string, boolean, ...

→ if no return value → void.

④ → method name

⑤ → List of parameters. (formal parameters.)

(12) Parameter → It's like a placeholder,
when a method is invoked, you pass
a value to the parameter.

for ex → $\text{int } z = \text{sum}(x, y)$

↙ ↘
when we invoke the method, actual parameters (arguments)

Note → Parameters are optional →
a method doesn't have to contain
any parameters.

ex → $\text{math.random}()$; has no
parameters

(14) $\text{math.random}()$ $\rightarrow$ generates a pseudo-random double value that is greater than or equal to 0.0 and strictly less than 1.0.

$\rightarrow$ min value $\rightarrow$ 0.0 (inclusive)

$\rightarrow$ max possible value $\rightarrow$ 0.99999

(exclusive it will never return exactly 1.0).

$\rightarrow$ To want a whole no. -

$\text{int randomNumber} = (\text{int})(\text{math.}$

$\text{random}() * 10) + 1;$

$\rightarrow * 10$ scales the range up from

0.0 to 0.9999...

(before it was 0 to 0.999...)

$\rightarrow (\text{int})$ $\rightarrow$ truncates the decimals completely

-ely

$+1$ $\rightarrow$ shifts the entire range up by one.

(15) So general formula -

int generate(int) (Math.random() * (max - min + 1)) + min;

Q.7. if want to generate b/w 5 and 15

So $\rightarrow$ Calculate the range
 $(\text{max} - \text{min} + 1) = (15 - 5 + 1) = 11$

$\Rightarrow (\text{Math.random()} * 11) + 5$

$\downarrow$
range $\rightarrow (0.0, 11.0)$

int(0.0, 10.0)

(0, 10)

shift up adding the min value

range $\rightarrow (5, 15)$

Calling a method - (16)

→ to execute a method you have to call or invoke a method.

→ two ways to do that -

(1) `int larger = max(3, 4);`
→ assigns the result of the method to the variable

(2) `System.out.println(max(3, 4));`
→ which prints the return value of the method ~~to~~ `max(3, 4)`.

Note → If a method returns void, the call to the method must be a statement.

Working behind methods

→ The System creates an activation record (also called activation for that stores parameters and variables for the method
→ places the activation record in the area of memory known as call stack.

Working - The call stack stores the activation records in a last in - first out fashion.

①

Space for main method
var:
j = 30
i = 20

→

Space for main method
var:
j = 30
i = 20
Space for minimum method
result: 30
a: 20
b: 30

(a) main method is invoked

(b) minimum method is invoked

✱

(18)

Space required
for main method
for main 30
p=30
q=20

(c) The main
method is finished
and return value
to main

Stack is
empty.

(d) The main
method is
finished

Void method —

does not return a value.

→ A return statement is not needed for
a void method, but it can be used
for terminating the method and
returning to the method's call.

19) Types of Methods

① Instance Method - that belongs to

→ A method of a class - an object of a class to

→ You need to create an object to call it.

Ex → Class Student {

void display() {

System.out.print(" -

Instance method with void return type.

public static void main (String

args) {

Student s = new Student

s.display();

}

Class name

this calls the display method in Student class.

20

① Student s = new Student();

→ This whole line means we are creating a Student class object

~~using new~~

→ new Student(); → will initialize the object.

→ s → reference variable through

which we can call any variable, method

② Static method -

A method that belongs to the class rather than an individual object.

→ declared using static.

```
class Student {  
    static void display() {
```

```
        S.O.P(" — ");
```

```
    }  
    public static void main(String[] args) {
```

```
        Student.display();
```

```
    } // or just → display;
```


④ Final Method -

A method declared with ~~final~~ final keyword cannot be overridden by a child class.

ex-7 Class Animal {
 final void eat() {
 SOP("___");
 }

→ A child class cannot use the same method name, ~~at~~ ^{not} in the same class.

for ex -> Class Dog extends Animal {

This will cause an error {
 void eat() {
 SOP("___");
 }

(23)

Using Static and Instance methods with static and instance variables

C.7

```
public class Car {
```

```
    String model;
```

```
    int year;
```

```
    public static void main (String[] args)
```

```
    {
```

```
        Car c1 = new Car();
```

```
        c1.model = "Honda";
```

```
        c1.year = 123;
```

```
        Car c2 = new Car();
```

```
        c2.model = "Hyundai";
```

```
        c2.year = 124;
```

```
    }
```

creating objects for

initializing

instance variables

printing

here only

o/p -> Honda 123

Hyundai 124

```
System.out.println(c1.model + " " + c1.year);
```

```
System.out.println(c2.model + " " + c2.year);
```

Q2. What if I invoke a method in place of printing in main ~~method~~ only.

ex → c1.display(); {
c2.display(); }

void display() → instance method

{
SOP(model + " " + year);
~~SOP(model + " " + year);~~
}

O/p → ~~Hyundai~~ Honda 123
Hyundai 124

⇒ Using Instance variable with instance method → we just have to invoke the method with respective ref. variable and we can multiple outputs.

Using Static method with static

variable -

Public class Car {

static String model;

static int year;

{

public static void main(

String[] args)

{

model = "Honda";

year = 123;

model = "Hyundai";

year = 124;

}

}

}

}

}

}

initializing
can be done
without obj.
but values
are overwritten

if also using
objects
values are
overwritten.

Car c1 = new Car();

c1.model = "Honda";

c1.year = 123;

Car c2 = new Car();

c2.model = "Hyundai";

c2.year = 124;

Q/P

7 Honda

7

7

7

7

7

7

7

7

7

7

7

7

7

7

7

7

7

Q/P { System.out.print(C1.model + C1.gcc);
 → Honda 123 } System.out.print(C2.model + C2.gcc);

⇒ if using method to print the values -
 display(); } we can directly
 invoke static

methods.

static void display() { these were
 declared
 as static
 SOP(model + gcc); }

but we can't use instance variable if this using static method. } we can directly print or
 can use object also.

or static void display()

{
 Car C1 = new Car();
 Car C2 = new Car();
 SOP(C1.model + C2.gcc);
 SOP(C2.model + C2.gcc);
 }

guess the o/p →

→ Honda 123 + Honda 123

(27)

Constructors

A constructor contains an initialization code but is not a method.

- > Has same name as the class
- > Has no return type
- > Is automatically called when an object is created.

-> Student s = new Student();

Types

①

Java automatically creates a constructor

②

-> called as default constructor

for ex -> public class Car {

String model;

int year;

public static void main

(String[] args)

(28)

28

```
{  
    Car c1 = new Car();  
    SOP(c1.model + " " + c1.yr);  
}
```

O/P → null 0

→ the default constructor initializes default values according to the data types.

Types of Constructors —

automatically

① Default → created when not created externally

② Parametrized — when we pass arguments

— created when the object is initialized

Car → Car c1 = new Car("Honda", 123);

Car (String s, int yr)

model = s;
yr = yr;

(29)

③ No-argument Constructor

— when we internally made a constructor but didn't pass the argument.

ex -> Car c1 = new Car();

```
Car c1;
```

```
{
```

```
model = "not decided";
```

```
car = 0;
```

```
}
```

```
Car Constructor
```

We can just add random values to the variables, just in case we have not decided values.

Constructor Overloading -

just like method overloading

- Same name

- Same class

- Different no. of parameter / or different parameter

30) Constructor Chaining

The process of calling one constructor from another constructor either within the same class or from a parent class.
via `this()` or `super()`

Note → When ever we use `this()` or `super()` → they should very first statement

→ `this()` and `super()` cannot be used inside the same constructor.

For eg →

```
Car (String s, int x)
```

```
{  
    model = "Honda";  
    x = 5;  
}
```

```
Car C)
```

```
{  
    this ("Hyundai", 124);  
}
```

22

② → Its primary purpose is to pass data across multiple constructors to eliminate duplicate initialization code and manage code reusability.

Using super()

→ Class Person {

String name;

~~int age;~~

Person (String n) { Step 1

name = "Aakash";

super ("Person Constructor")

Step 3

}

Class Student extends Person {

int roll no;

Step 2 Student () { this (101, "Unknown")

```

22 Student (int x, String name) {
    step 3 super (name);
    small no = x; step 6
    SOP ("Student Constructor");
    }

```

```

    void display() { step 9
        SOP (name); no values
        SOP (small no);
    }

```

```

public class ConstructorChaining {
    public static void main (String[] args)

```

```

    {
        step 1 Student s = new Student();

```

```

        step 8 s.display();
    }

```

guess the ?
o/p

Step 1 -> No argument constructor of Student class is called.

-> this keyword will call the other constructor with argument

Using Access Modifiers

Keywords that determine the visibility and accessibility of classes, methods, constructors or variables.

Types of access modifiers -

- ① Public → ~~can~~ accessibility across different packages and different classes.
- ② Default → different classes but should lie in same package.
- ③ Private → Same class, same package.
- ④ Protected → Same package, different classes.

→ But the classes should be a subclass of a parent class.

34) Encapsulation - (Data hiding + Controlled access)

→ wrapping data and methods that operate on that data into a single class, while restricting direct access to the data.

1) making variables private

2) Providing public methods to access or modify them

using getter and setters

if() method name, then get method

isdef = two words

(if two words) private name then

private = one word

method